

ViBe System Architecture, Workflow & Complete Directory Analysis

1. System Overview & Core Philosophy

ViBe (*Video-based Intelligent Behavioral Analysis & Continuous Active Learning Platform*) is a comprehensive educational and assessment ecosystem inspired by the classical Indian tale of **Vikram and Betaal**. In the story, King Vikramaditya must answer Betaal's challenging riddles; any incorrect response causes Betaal to fly back, requiring Vikram to revisit the challenge. Similarly, ViBe enforces continuous active comprehension check-ins: when students falter during interactive video checkpoints or assessments, the platform prompts targeted reviews before allowing progression.

Key Architectural Pillars

1. **Frontend (`frontend/`)**: Built with **Vite, React (TypeScript), Tailwind CSS, and Shadcn UI**. It features rich interactive video playback, client-side AI monitoring (**whisperWorker**, gesture analysis), multi-type quiz rendering, emotion tracking, and comprehensive teacher/student dashboards.
2. **Backend (`backend/`)**: Built with **Express, InversifyJS (Dependency Injection), MongoDB (with Repository/Service/Controller pattern), and Firebase Authentication**. It enforces Role-Based Access Control (RBAC), manages course graphs, calculates watch-time and anomaly metrics, handles AI proctoring policies, and integrates with GenAI models.
3. **CLI (`cli/`)**: A dedicated command-line automation tool (**vibe CLI**) for local development setup, Firebase emulator synchronization, MongoDB binary preloading, and diagnostic checks.
4. **Documentation (`docs/`)**: Built on **Docusaurus** to provide developer guides, API references (via integrated Scalar OpenAPI docs), and onboarding resources.
5. **Quality Assurance (`e2e/`)**: End-to-End testing powered by **Playwright** simulating full student and teacher workflows.

2. End-to-End System Workflows

```
graph TD
    subgraph Client [Frontend App - Vite/React]
        UI[User Interface & Dashboards]
        VidPlayer[Interactive Video & Checkpoints]
        Workers[Background Workers: Whisper & Gesture]
        State[Zustand & Context Stores]
    end

    subgraph API [Backend Server - Express / Inversify DI]
        Routes[Routing Controllers & Middleware]
        AuthMod[Auth & RBAC Module]
        CourseMod[Courses & Quizzes Module]
    end
```

```

    ProctorMod[Anomalies, HP & Ejection Module]
    EmotionMod[Emotion & Sentiment Analytics Module]
    GenAIMod[GenAI & SmartBloom Module]
end

subgraph Data & Services [External & Persistence Layer]
    FB[Firebase Auth & Emulators]
    Mongo[(MongoDB Database)]
    ONNX[Client-Side ONNX AI Models]
    Sentry[Sentry Error & Performance Logging]
end

UI -->|JWT / API Calls| Routes
VidPlayer -->|Checkpoints & Watch Time| CourseMod
VidPlayer -->|Audio / Video Frames| Workers
Workers -->|Local Inference| ONNX
UI -->|Emotion Submissions| EmotionMod
Routes --> AuthMod & CourseMod & ProctorMod & EmotionMod & GenAIMod
AuthMod --> FB
CourseMod & ProctorMod & EmotionMod --> Mongo
API --> Sentry

```

A. Authentication & Role-Based Access Control (RBAC) Workflow

6. **User Login/Signup:** Users authenticate via [LoginPage.tsx](#) using Firebase Authentication (Email/Password or OAuth).
7. **Token Verification & Sync:** The frontend obtains a Firebase ID Token and calls `POST /api/auth/login`. The backend ([FirebaseAuthService.ts](#)) verifies the token, syncs or creates the user in MongoDB, and assigns roles (`student`, `teacher`, `admin`).
8. **Route Protection:** [protected-route.tsx](#) and backend guards (`@Authorized()`, [AbilityDecorator.ts](#)) enforce access control based on user roles and granular permissions.

B. Course Delivery & Vikram-Betaal Active Learning Loop

9. **Course Structure:** Courses are hierarchical: `Course` -> `Version` -> `Module` -> `Section` -> `Item` (`Video`, `Quiz`, or `Article`).
10. **Interactive Video Playback:** [video.tsx](#) streams video and monitors playback time. At predefined timestamps, the video pauses and presents embedded check-for-understanding quizzes.
11. **Adaptive Remediation:** If a student fails an in-video checkpoint, the system prevents skipping ahead and triggers the Vikram-Betaal review loop, directing the student back to the specific video segment where the concept was taught.

C. AI Proctoring, Anomaly Tracking & Heart Points (HP) Workflow

12. **Client-Side AI Monitoring:** During high-stakes assessments, browser-based workers ([whisperWorker.js](#), gesture tracking) and webcam captures run locally using converted ONNX models ([convert_segbot_to_onnx.py](#), [convert_whisper_to_onnx.py](#)).
13. **Anomaly Reporting:** If suspicious behavior occurs (tab switching, multiple faces, voice detection, prolonged looking away), the client dispatches flags (`/api/anomalies`).

14. **HP (Heart Points) & Ejection Policy:** Each student starts with a set number of Heart Points. Anomalies deduct HP based on course proctoring policies ([/api/ejection-policy](#)). If HP drops to zero, the student is automatically locked out ([EjectionPolicyService](#)).

D. Learner Emotion & Sentiment Analytics Workflow

15. **Learner Check-in:** Students submit their current emotional state using [EmotionSelector.tsx](#) ([very_sad](#), [sad](#), [neutral](#), [happy](#), [very_happy](#)).
16. **Backend Aggregation:** The backend [emotions](#) module stores timestamped entries linked to the specific course item.
17. **Teacher Insights:** Teachers view real-time sentiment distribution, drop-off correlates, and individual learner emotion trajectories on [EmotionAnalyticsDashboard.tsx](#) and [StudentEmotionJourney.tsx](#).

E. GenAI & SmartBloom Pedagogical Generation

18. **AI Workflow & SmartBloom:** Teachers can utilize AI assistants ([SmartBloomWorkflow.tsx](#), [AiWorkflow.tsx](#)) to generate course outlines, formulate Bloom's Taxonomy-aligned quiz items ([SOL](#), [SML](#), [MTL](#), [OTL](#), [NAT](#), [DES](#)), and automatically evaluate subjective descriptive responses.

3. Complete Directory & File-by-File Analysis

Root Workspace Files

File Name	Purpose & Detailed Explanation
README.md	High-level project overview explaining the Vikram & Betaal inspiration, active learning philosophy, and main features.
LICENSE	MIT License covering the entire codebase.
package.json	Root package definition configuring pnpm workspaces and top-level utility scripts (postinstall , build , etc.).
pnpm-workspace.yaml	Defines the multi-package monorepo workspace containing backend , frontend , cli , and docs .
pnpm-lock.yaml	Exact dependency lockfile guaranteeing deterministic installs across all monorepo packages.
firebase.json & .firebaserc	Firebase configuration linking local emulator ports, hosting targets, and cloud project aliases.

.gitignore & .gitattributes	Specifies ignored files (build artifacts, <code>.env</code> , <code>node_modules</code>) and line-ending normalization (<code>LF/CRLF</code>).
setup.py	Python package installer setup script used when packaging optional Python helper utilities or AI scripts.
convert_segbot_to_onnx.py	PyTorch script defining the SEGBOT encoder-decoder-pointer architecture and exporting weights to ONNX format for browser inference.
convert_whisper_to_onnx.py	Script utilizing HuggingFace Optimum to export <code>openai/whisper-base</code> into an ONNX format for client-side speech processing.
EMOTION_TRACKING_IMPLEMENTATION.md	Architectural documentation detailing how learner emotion tracking and sentiment charts are structured across frontend and backend.
EMOTION_ANALYTICS_INTEGRATION.md	Integration guide explaining how teacher analytics dashboards read aggregated emotion data from API endpoints.
EMOTION_QUICK_REFERENCE.md	Quick developer reference summarizing the 5 emotion states, API schemas, and UI components.
FINAL_INTEGRATION_CHECKLIST.md	Quality assurance checklist for verifying module integration across authentication, courses, proctoring, and UI before deployment.

Backend (`backend/`)

The server uses **InversifyJS** for clean dependency injection, **Routing-Controllers** for decorator-based routing, and **MongoDB** for storage.

Configuration, Bootstrap & Entry (`backend/src/`)

File Path	Purpose & Detailed Explanation
src/index.ts	Main entry point initializing Sentry, connecting to MongoDB, loading Inversify modules, and starting the Express server.
src/container.ts	Root DI container assembly tying together repositories, services,

	controllers, and shared database providers.
src/types.ts	Defines unique Symbol identifiers used by Inversify (TYPES.Database , TYPES.CourseRepository , etc.) to bind dependencies.
src/instrument.ts	Sentry profiling and telemetry configuration loaded before app initialization to capture uncaught errors and trace performance.
src/inversify-adapter.ts	Custom IocAdapter allowing routing-controllers to resolve controller instances directly from the Inversify container.
src/bootstrap/loadModules.ts	Dynamic module loader scanning all directories under src/modules/ and registering their controllers and services into the container.
src/config/app.ts	Core application configuration (server port, CORS whitelist, API prefix, environment flags).
src/config/db.ts	MongoDB connection strings and database options parsed from environment variables.
src/config/ai.ts	Configuration for external GenAI APIs (API keys, model parameters for OpenAI/Gemini/Anthropic).
src/config/sentry.ts , smtp.ts , storage.ts	Configs for error tracking DSNs, SMTP transactional email servers, and cloud storage bucket connections.

Shared Layer (`backend/src/shared/`)

Folder / File Path	Purpose & Detailed Explanation
shared/classes/BaseService.ts	Abstract base service class providing common logging, error handling, and transactional wrappers for business logic.
shared/constants/constants.ts	System-wide constants (HTTP status codes, default pagination limits, role definitions, proctoring threshold defaults).
shared/database/providers/mongo/MongoDatabase.ts	Singleton MongoDB database connection manager establishing pool connections and handling reconnection logic.

shared/database/providers/mongo/repositories/	Concrete MongoDB implementations for all data contracts (<code>CourseRepository.ts</code> , <code>UserRepository.ts</code> , <code>EnrollmentRepository.ts</code> , <code>ProgressRepository.ts</code> , <code>ItemRepository.ts</code> , <code>SettingRepository.ts</code> , <code>AppealRepository.ts</code> , <code>AnnouncementRepository.ts</code> , <code>SlotBookingRepository.ts</code> , <code>UserActivityEventRepository.ts</code>).
shared/functions/generateOpenApiSpec.ts	Dynamically introspects <code>routing-controllers</code> decorators and class-validator schemas to build a complete OpenAPI 3.0 specification.
shared/functions/authorizationChecker.ts & currentUserChecker.ts	Middleware callbacks executed by <code>routing-controllers</code> to validate JWTs, check role requirements, and inject the authenticated user into controller methods.
shared/functions/AbilityDecorator.ts	Parameter decorator enabling fine-grained attribute-based access checks within endpoint signatures.
shared/functions/verifyRecaptcha.ts	Validates Google reCAPTCHA tokens to prevent bot registrations and automated brute-force attacks.
shared/interfaces/*.ts	TypeScript contract definitions (<code>ICourseRepository</code> , <code>IUserRepository</code> , <code>IAppeal</code> , <code>ISettingRepository</code> , etc.) ensuring strict dependency inversion.
shared/middleware/errorHandler.ts	Global error interception handler translating thrown domain exceptions and validation errors into uniform JSON error responses.
shared/middleware/ApiKeyAuthMiddleware.ts , corsHandler.ts , loggingHandler.ts , rateLimiter.ts	Express middlewares handling API key authentication for background jobs, CORS headers, HTTP request logging, and rate-limiting against denial of service.

Domain Modules (``backend/src/modules/``)

Each module encapsulates its own controllers, services, repositories, models, and container bindings.

- ``auth/``: Handles authentication workflows, user onboarding, password changes, and Firebase Admin verification (`AuthController`, `FirebaseAuthService`).
- ``courses/``: Manages the complete curriculum hierarchy (Courses, Versions, Modules, Sections, Items), publishing workflows, and item sequencing (`CourseController`, `CourseService`).
- ``quizzes/``: Evaluates student submissions across 6 distinct question formats (`SOL`, `SML`, `MTL`, `OTL`, `NAT`, `DES`), calculates scores, and stores question pools (`QuizController`, `QuizService`).

- **`users/`**: Tracks enrollments, calculates cumulative video watch-time, computes progress percentages, and updates user profile data (**UserController**, **EnrollmentService**, **ProgressService**).
- **`emotions/`**: Records real-time emoji check-ins (**very_sad** to **very_happy**), computes item sentiment averages, and serves emotion analytics for teacher dashboards (**EmotionController**, **EmotionService**).
- **`anomalies/`** & **`ejectionPolicy/`** & **`hpSystem/`**: Enforces academic integrity. Logs AI proctoring flags (multiple faces, voice detected, tab switches), calculates deductions from Heart Points (HP), and locks out students violating policy thresholds (**AnomalyController**, **EjectionPolicyService**).
- **`genAI/`**: Provides AI generation capabilities for generating course outlines, formulating Bloom's Taxonomy-based quiz questions, and generating descriptive feedback (**GenAIController**, **GenAIService**).
- **`notifications/`** & **`announcements/`**: Handles system-wide course announcements, email invitations, and transactional notification dispatch (**NotificationController**, **AnnouncementService**).
- **`settings/`**: Manages global platform configuration and course-specific proctoring requirements (**SettingController**, **SettingService**).
- **`courseRegistration/`**: Manages student enrollment requests, teacher approvals/rejections, and cohort allocations (**CourseRegistrationController**).
- **`studentQuestions/`**: Allows students to submit doubts/questions attached to specific course timestamps for teacher review (**StudentQuestionController**).
- **`auditTrails/`** & **`reports/`**: Logs sensitive administrative actions and generates CSV/Excel export datasets for institutional reporting (**AuditTrailController**, **ReportService**).

Workers, Utilities & Plop Scaffolding (`backend/src/workers/`, `src/utils/`, `plop-templates/`)

File Path	Purpose & Detailed Explanation
src/workers/backup-cron.ts & startCron.ts	Background cron job scheduler running automated daily MongoDB database snapshots and cleanup routines.
src/workers/clone-course*.ts & invite-email*.ts	Node.js Worker Thread pools handling heavy background tasks (deep-cloning large course trees and batch sending invitation emails without blocking the main event loop).
src/utils/logDetails.ts	Prints an ASCII startup banner and a cleanly formatted table of all active API routes when the server launches.
src/utils/env.ts , chunkArray.ts , toISOFormat.ts	Helper functions for type-safe environment variable lookup, array batching, and strict ISO date formatting.

plopfile.cjs & plop-templates/	Scaffolding templates (controller.hbs , service.hbs , repository.hbs) enabling developers to generate fully wired domain modules instantly via CLI.
scripts/generate-openapi.cjs	Build-time script exporting the generated OpenAPI JSON schema to disk for documentation generation.
Dockerfile & Dockerfile-all	Multi-stage container build definitions for standalone backend deployment or all-in-one full-stack builds.

Frontend (`frontend/`)

Built with **Vite**, **React (TypeScript)**, **Tailwind CSS**, and structured for modular client-side rendering.

Core Boot, Routing & Configuration (`frontend/src/`)

File Path	Purpose & Detailed Explanation
src/main.tsx	Root application mount point binding the React tree to index.html within strict mode providers.
src/app.tsx	Top-level application component wrapping global context providers (Theme, Auth, Toast notifications, Sentry).
src/app/router.tsx	Central React Router configuration declaring public pages, student portals, and protected teacher/admin routes.
src/app/routes/student-routes.tsx & teacher-routes.tsx	Route definitions separated cleanly by user persona (/student/dashboard , /teacher/courses , etc.).

Pages (`frontend/src/app/pages/`)

Folder / File Path	Purpose & Detailed Explanation
pages/LoginPage.tsx , auth-page.tsx , ForgotPasswordPage.tsx , ResetPasswordPage.tsx	Comprehensive authentication interface handling sign-in, user registration, role selection, and secure password reset via Firebase.

SelectRolePage.tsx	
pages/student/dashboard.tsx & courses.tsx	Student home view displaying enrolled courses, overall completion percentages, recent activity, and quick-continue links.
pages/student/course-page.tsx	The core learning workspace displaying the curriculum sidebar, rendering interactive video checkpoints, quizzes, and articles.
pages/student/CourseRegistration.tsx	Allows students to browse available courses and submit registration requests to instructors.
pages/student/leaderboard.tsx & components/LeaderboardLeagues.tsx	Gamified leaderboard ranking students by XP, quiz accuracy, and completed watch time across league tiers.
pages/student/FlagResponse.tsx	Interface allowing students to review automated proctoring flags and submit appeals explaining potential false positives.
pages/teacher/dashboard.tsx & teacher-course-page.tsx	Teacher command center for curriculum authoring, module organization, item creation, and section management.
pages/teacher/course-enrollments.tsx & CourseRegistrationRequests.tsx	Displays enrolled students, allows approving/rejecting enrollment requests, and manages student access.
pages/teacher/FlaggedList.tsx & AnomaliesList.tsx	Security monitoring dashboard listing all proctoring violations, timestamps, and evidence snapshots for instructor review.
pages/teacher/AiWorkflow.tsx , SmartBloomWorkflow.tsx , AISectionPage.tsx	Advanced pedagogical workspaces utilizing LLMs to automatically generate course structures and Bloom's Taxonomy-aligned assessments.
pages/teacher/ejection-policies.tsx & configure-cohorts.tsx	Allows teachers to configure Heart Point rules, anomaly deduction thresholds, and group students into learning cohorts.
pages/shared/NotificationsPage.tsx	Unified notification inbox displaying course updates, grading completions, and invitation alerts.

Core Components & UI Library (``frontend/src/components/``)

File Path	Purpose & Detailed Explanation
components/video.tsx	Highly custom interactive video player supporting precise timestamp check-ins, playback speed control, and checkpoint pauses.
components/quiz.tsx	Multi-format quiz rendering and submission engine supporting SOL (Single Option), SML/MTL (Multiple Option), OTL (Ordering), NAT (Numerical), and DES (Descriptive) questions.
components/article.tsx	Rich markdown content renderer supporting code syntax highlighting, embedded math equations, and read-time tracking.
components/EmotionSelector.tsx	Interactive emoji bar allowing learners to log their emotional response during or after finishing a lesson.
components/EditProctoringModal.tsx	Configuration dialog enabling instructors to toggle webcam checks, tab-switch penalties, and audio monitoring for quizzes.
components/floating-video.tsx	Picture-in-picture floating video component allowing students to continue watching lessons while browsing supplementary materials.
components/question-upload-dialog.tsx	Bulk import dialog supporting CSV/Excel spreadsheets to upload dozens of quiz items simultaneously.
components/StudentQuestionComposer.tsx	Input modal enabling students to attach timestamped questions or clarification notes directly to specific moments in a video.
components/app-sidebar.tsx , nav-*.tsx , layout/	Core navigation structure providing collapsible sidebars, breadcrumbs, user profile dropdowns, and responsive layouts.
components/ui/ *	Full suite of accessible, highly styled primitive UI components built using Radix UI and Tailwind CSS (Buttons, Dialogs, Selects, Cards, Tooltips, Tabs).

State Stores, Hooks & API Clients (``frontend/src/store/``, ``src/hooks/``, ``src/lib/``, ``src/utils/``)

File Path	Purpose & Detailed Explanation
-----------	--------------------------------

src/store/auth-store.ts	Zustand store persisting user identity, JWT tokens, active roles (student/teacher), and profile metadata across sessions.
src/store/course-store.ts & player-store.ts	State stores tracking the active course curriculum tree, current video playback timestamp, completed items, and active check-ins.
src/store/flag-store.tsx & anomaly-store.tsx	Stores maintaining active proctoring alerts, current remaining Heart Points (HP), and real-time security warnings.
src/hooks/hooks.ts	Comprehensive collection of custom React Query hooks wrapping every CRUD endpoint for courses, enrollments, and progress.
src/hooks/use-emotion.ts & useTranscriber.ts	Custom hooks handling submission of learner emotion ratings and coordinating background client-side speech-to-text transcription.
src/lib/api-client.ts	Central Axios HTTP client preconfigured with base URLs, automatic JWT header injection, and global error interceptors.
src/lib/firebase.ts & genai-api.ts	Client-side initialization for Firebase authentication services and direct integration helpers for generative AI calls.
src/lib/excel-export.ts & policy-utils.ts	Utilities converting student progress tables into downloadable spreadsheets and calculating proctoring policy enforcement states.
src/types/schema.ts & *.types.ts	Frontend TypeScript type mirrors ensuring 1:1 parity with backend data structures across API boundaries.
src/workers/whisperWorker.js	Web Worker running ONNX-converted Whisper models inside the browser to detect speech without sending audio streams to a server.
vite.config.ts , index.html , nginx.conf	Vite bundler build settings, base HTML container with font imports, and production Nginx reverse-proxy configuration.

CLI Tool (`cli/`)

The **vibe** command-line tool automates developer workflow setups and environment orchestration.

File Path	Purpose & Detailed Explanation
-----------	--------------------------------

cli/src/cli.ts	Entry point using commander to register available CLI commands (setup , start , test , help).
cli/src/findRoot.ts	Utility traversing upwards from the current directory to locate the root monorepo directory containing pnpm-workspace.yaml .
cli/src/commands/setup.ts , start.ts , test.ts	Command executors orchestrating full environment bootstrapping, running local dev servers, and kicking off test suites.
cli/src/steps/env.ts , firebase-emulators.ts , mongodb-binary.ts	Discrete, modular setup tasks checking .env variables, launching local Firebase emulators, and downloading embedded MongoDB binaries.

Documentation (`docs/`) & End-to-End Tests (`e2e/`)

Folder / File Path	Purpose & Detailed Explanation
docs/docusaurus.config.ts & sidebars.ts	Docusaurus site configuration, navigation bar setup, theme customization, and hierarchical sidebar structure for documentation.
docs/docs/ & docs/newdocs/	Markdown documentation files covering System Architecture, API integration, onboarding tutorials, and feature explainers.
e2e/playwright.config.ts	Playwright test runner configuration defining headless browser options, base URLs, timeouts, and report generation settings.
e2e/tests/smoke.spec.ts	Basic smoke verification test ensuring that the login page and primary application shell load cleanly without console errors.
e2e/tests/play-course-videos.test.ts & test-progress-status.test.ts	Comprehensive E2E suites automating video playback, simulating checkpoint answer submissions, and verifying progress updates.
e2e/tests/common-utils.ts	Shared automation utilities for logging into test accounts, seeding test courses in MongoDB, and navigating complex UI states.
scripts/setup-win.ps1 & setup-	Cross-platform shell scripts (PowerShell and Bash) automating

unix.sh	Node/pnpm environment validation and initial project setup.
self-hosting/README.md	Guide detailing how to deploy ViBe using Docker containers, reverse proxies, and production MongoDB instances on self-hosted infrastructure.

4. Summary of System Mechanics

The entire repository operates as a tight, modular monorepo:

19. **Curriculum & Progress:** Courses are stored in MongoDB via structured schemas (`CourseRepository`, `ItemRepository`). When a student watches a lesson, `video.tsx` dispatches periodic watch-time heartbeats to `ProgressService`.
20. **Vikram & Betaal Loop:** If the video reaches a checkpoint timestamp, playback halts and `quiz.tsx` renders the question. Incorrect answers trigger an immediate seek backward (`Adaptive Remediation`), enforcing comprehension before progression.
21. **Proctoring & Integrity:** For high-stakes quizzes, `whisperWorker.js` and camera trackers run client-side ONNX models (`convert_*.py`). If anomalies occur, `AnomalyController` logs the violation, deducts points via `EjectionPolicyService`, and updates Heart Points (HP) in real time.
22. **Emotional Feedback:** Learners log check-ins via `EmotionSelector.tsx`, which are aggregated in `emotions` and displayed dynamically across `EmotionAnalyticsDashboard.tsx` to help instructors optimize course difficulty and pacing.